

Namespace Kouki.Mugura.Attributes

Classes

[CommandAttribute](#)

Enums

[CommandCategory](#)

Class CommandAttribute

Namespace: [Kouki.Mugura.Attributes](#)

Assembly: Kouki.Mugura.dll

```
[AttributeUsage(AttributeTargets.Class, Inherited = false)]  
public sealed class CommandAttribute : Attribute
```

Inheritance

[object](#)  ← [Attribute](#)  ← CommandAttribute

Inherited Members

[Attribute.Equals\(object\)](#)  , [Attribute.GetCustomAttribute\(Assembly, Type\)](#)  ,
[Attribute.GetCustomAttribute\(Assembly, Type, bool\)](#)  ,
[Attribute.GetCustomAttribute\(MemberInfo, Type\)](#)  ,
[Attribute.GetCustomAttribute\(MemberInfo, Type, bool\)](#)  ,
[Attribute.GetCustomAttribute\(Module, Type\)](#)  , [Attribute.GetCustomAttribute\(Module, Type, bool\)](#)  ,
[Attribute.GetCustomAttribute\(ParameterInfo, Type\)](#)  ,
[Attribute.GetCustomAttribute\(ParameterInfo, Type, bool\)](#)  , [Attribute.GetCustomAttributes\(Assembly\)](#)  ,
[Attribute.GetCustomAttributes\(Assembly, bool\)](#)  , [Attribute.GetCustomAttributes\(Assembly, Type\)](#)  ,
[Attribute.GetCustomAttributes\(Assembly, Type, bool\)](#)  , [Attribute.GetCustomAttributes\(MemberInfo\)](#)  ,
[Attribute.GetCustomAttributes\(MemberInfo, bool\)](#)  ,
[Attribute.GetCustomAttributes\(MemberInfo, Type\)](#)  ,
[Attribute.GetCustomAttributes\(MemberInfo, Type, bool\)](#)  , [Attribute.GetCustomAttributes\(Module\)](#)  ,
[Attribute.GetCustomAttributes\(Module, bool\)](#)  , [Attribute.GetCustomAttributes\(Module, Type\)](#)  ,
[Attribute.GetCustomAttributes\(Module, Type, bool\)](#)  , [Attribute.GetCustomAttributes\(ParameterInfo\)](#)  ,
[Attribute.GetCustomAttributes\(ParameterInfo, bool\)](#)  ,
[Attribute.GetCustomAttributes\(ParameterInfo, Type\)](#)  ,
[Attribute.GetCustomAttributes\(ParameterInfo, Type, bool\)](#)  , [Attribute.GetHashCode\(\)](#)  ,
[Attribute.IsDefaultAttribute\(\)](#)  , [Attribute.IsDefined\(Assembly, Type\)](#)  ,
[Attribute.IsDefined\(Assembly, Type, bool\)](#)  , [Attribute.IsDefined\(MemberInfo, Type\)](#)  ,
[Attribute.IsDefined\(MemberInfo, Type, bool\)](#)  , [Attribute.IsDefined\(Module, Type\)](#)  ,
[Attribute.IsDefined\(Module, Type, bool\)](#)  , [Attribute.IsDefined\(ParameterInfo, Type\)](#)  ,
[Attribute.IsDefined\(ParameterInfo, Type, bool\)](#)  , [Attribute.Match\(object\)](#)  , [Attribute.TypeId](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetType\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Constructors

CommandAttribute(string)

```
public CommandAttribute(string name)
```

Parameters

name [string](#)

Properties

Category

```
public CommandCategory Category { get; set; }
```

Property Value

[CommandCategory](#)

Description

```
public string? Description { get; set; }
```

Property Value

[string](#)

Example

```
public string? Example { get; set; }
```

Property Value

[string](#)

Name

```
public string Name { get; }
```

Property Value

[string](#)

RequiresConfirmation

```
public bool RequiresConfirmation { get; set; }
```

Property Value

[bool](#)

Enum CommandCategory

Namespace: [Kouki.Mugura.Attributes](#)

Assembly: Kouki.Mugura.dll

[Flags]

```
public enum CommandCategory
```

Fields

Admin = 0

Dangerous = 0

Environment = 0

FileOperations = 0

Git = 0

Integration = 0

Navigation = 0

Terminal = 0

Uncategorized = 0

Namespace Kouki.Mugura.Commands

Classes

[TestCommand](#)

Class TestCommand

Namespace: [Kouki.Mugura.Commands](#)

Assembly: Kouki.Mugura.dll

```
[Command("test", Category = CommandCategory.Admin, Description = "just a test command",  
RequiresConfirmation = false)]  
public record TestCommand : ICommand, IEquatable<TestCommand>
```

Inheritance

[object](#)  ← TestCommand

Implements

[ICommand](#), [IEquatable](#)  <[TestCommand](#)>

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

TestCommand(string?)

```
public TestCommand(string? Value)
```

Parameters

Value [string](#) 

Properties

Value

```
public string? Value { get; init; }
```

Property Value

[string](#) 

Namespace Kouki.Mugura.Dependency Injection

Classes

[ServiceCollectionExtensions](#)

Методы расширения для регистрации ядра Mugura в DI-контейнере.

Class ServiceCollectionExtensions

Namespace: [Kouki.Mugura.DependencyInjection](#)

Assembly: Kouki.Mugura.dll

Методы расширения для регистрации ядра Mugura в DI-контейнере.

```
public static class ServiceCollectionExtensions
```

Inheritance

[object](#) ← ServiceCollectionExtensions

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Remarks

Метод [AddMugura\(IServiceCollection, params Assembly\[\]\)](#) регистрирует все необходимые компоненты:

- [CommandRegistry](#) — синглтон.
- [ICommandFactory](#) (реализация [CommandFactory](#)) — синглтон.
- [IScopeManager](#) (реализация [ScopeManager](#)) — синглтон.
- [ICommandDispatcher](#) (реализация [CommandDispatcher](#)) — синглтон.
- [IRouter](#) (реализация [StaticRouter](#)) — синглтон.
- Открытый generic [ICommandValidator<TCommand>](#) → [DataAnnotationsValidator<TCommand>](#).
- Все классы, реализующие [ICommandHandler<TCommand>](#), из указанных сборок регистрируются как Scoped.

Если сборки не указаны, используется вызывающая сборка (assembly вызывающего метода).

После вызова этого метода можно переопределить валидатор для конкретной команды, зарегистрировав свою реализацию [ICommandValidator<TCommand>](#).

Methods

AddMugura(IServiceCollection, params Assembly[])

Регистрирует ядро Mugura в DI контейнере.

```
public static IServiceCollection AddMugura(this IServiceCollection services, params
Assembly[] commandHandlerAssemblies)
```

Parameters

services [IServiceCollection](#) 

Коллекция сервисов.

commandHandlerAssemblies [Assembly](#)  []

Сборки, в которых нужно искать хендлеры команд (если не указаны, используется вызывающая сборка).

Returns

[IServiceCollection](#) 

IServiceCollection для цепочки вызовов.

Namespace Kouki.Mugura.Dispatcher

Classes

[CommandDispatcher](#)

todo

[DelegatingRouter](#)

Базовый класс для роутеров-обёрток, которые делегируют выполнение другому роутеру.

Interfaces

[ICommandDispatcher](#)

[IRouter](#)

Class CommandDispatcher

Namespace: [Kouki.Mugura.Dispatcher](#)

Assembly: Kouki.Mugura.dll

todo

```
public class CommandDispatcher : ICommandDispatcher
```

Inheritance

[object](#) ← CommandDispatcher

Implements

[ICommandDispatcher](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Constructors

CommandDispatcher(IScopeManager)

```
public CommandDispatcher(IScopeManager scopeManager)
```

Parameters

scopeManager [IScopeManager](#)

Methods

SendAsync<TCommand>(TCommand, IExecutionContext, CancellationToken)

```
public Task<JsonNode> SendAsync<TCommand>(TCommand command, IExecutionContext context, CancellationToken cancellationToken) where TCommand : ICommand
```

Parameters

command `TCommand`

context [IExecutionContext](#)

cancellationToken [CancellationToken](#)

Returns

[Task](#) [JsonNode](#) >

Type Parameters

TCommand

Class DelegatingRouter

Namespace: [Kouki.Mugura.Dispatcher](#)

Assembly: Kouki.Mugura.dll

Базовый класс для роутеров-обёрток, которые делегируют выполнение другому роутеру.

```
public abstract class DelegatingRouter : IRouter
```

Inheritance

[object](#)  ← DelegatingRouter

Implements

[IRouter](#)

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Examples

```
public class IntentRouter : DelegatingRouter
{
    private readonly IIntentClassifier _classifier;

    public IntentRouter(IRouter innerRouter, IIntentClassifier classifier)
        : base(innerRouter)
    {
        _classifier = classifier;
    }

    public override async Task<JsonNode> RouteAsync(string commandName, JsonNode? argument,
Cancellation token ct)
    {
        // Преобразуем голосовую фразу в имя команды и аргументы
        var (name, args) = await _classifier.ClassifyAndExtract(commandName, ct);
        return await base.RouteAsync(name, args, ct);
    }
}
```

Remarks

Используйте этот класс, когда ваш роутер должен выполнить дополнительную обработку (например, классификацию интента, кеширование, удалённый вызов), а затем передать управление внутреннему роутеру (обычно [StaticRouter](#)).

Constructors

DelegatingRouter(IRouter)

Инициализирует новый экземпляр класса [DelegatingRouter](#).

```
protected DelegatingRouter(IRouter innerRouter)
```

Parameters

innerRouter [IRouter](#)

Внутренний роутер, которому будет передано выполнение.

Exceptions

[ArgumentNullException](#) 

Если **innerRouter** равен null.

Fields

InnerRouter

Внутренний роутер, которому делегируется выполнение. TODO: переписать комментарии под context

```
protected readonly IRouter InnerRouter
```

Field Value

[IRouter](#)

Methods

RouteAsync(string, JsonNode?, IExecutionContext?, Cancellation Token)

Выполняет маршрутизацию команды. По умолчанию делегирует вызов внутреннему роутеру.

```
public virtual Task<JsonNode> RouteAsync(string commandName, JsonNode? argument,
    IExecutionContext? context = null, CancellationToken cancellationToken = default)
```

Parameters

commandName [string](#)

Имя команды (или фраза для IntentRouter).

argument [JsonNode](#)

Аргументы команды в формате JsonNode.

context [IExecutionContext](#)

cancellationToken [CancellationToken](#)

Токен отмены.

Returns

[Task](#) <[JsonNode](#)>

Результат выполнения команды в виде JsonNode.

Interface ICommandDispatcher

Namespace: [Kouki.Mugura.Dispatcher](#)

Assembly: Kouki.Mugura.dll

```
public interface ICommandDispatcher
```

Remarks

TODO: Добавить параметр `configureScope` для передачи контекста в скоуп. Это позволит роутерам настраивать Scoped-сервисы (например, `IContext`) до резолва хендлера.

Methods

`SendAsync<TCommand>(TCommand, IExecutionContext, CancellationToken)`

```
Task<JsonNode> SendAsync<TCommand>(TCommand command, IExecutionContext context,  
CancellationToken cancellationToken = default) where TCommand : ICommand
```

Parameters

command TCommand

context [IExecutionContext](#)

cancellationToken [CancellationToken](#) 

Returns

[Task](#)  [<JsonNode](#)  >

Type Parameters

TCommand

Interface IRouter

Namespace: [Kouki.Mugura.Dispatcher](#)

Assembly: Kouki.Mugura.dll

```
public interface IRouter
```

Methods

RouteAsync(string, JsonNode?, IExecutionContext?, Cancellation Token)

```
Task<JsonNode> RouteAsync(string commandName, JsonNode? argument, IExecutionContext? context  
= null, CancellationToken cancellationToken = default)
```

Parameters

commandName [string](#) 

argument [JsonNode](#) 

context [IExecutionContext](#)

cancellationToken [CancellationToken](#) 

Returns

[Task](#)  < [JsonNode](#)  >

Namespace Kouki.Mugura.Factories

Classes

[CommandFactory](#)

Реализация [ICommandFactory](#), создающая DTO команды из [JsonNode](#)[↗].

Interfaces

[ICommandFactory](#)

Class CommandFactory

Namespace: [Kouki.Mugura.Factories](#)

Assembly: Kouki.Mugura.dll

Реализация [ICommandFactory](#), создающая DTO команды из [JsonNode](#).

```
public class CommandFactory : ICommandFactory
```

Inheritance

[object](#) ← CommandFactory

Implements

[ICommandFactory](#).

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Remarks

Для десериализации используется [System.Text.Json](#) с кешированием делегатов для каждого типа команды.

Если аргумент `argument` равен null, он заменяется на пустой [JsonObject](#).

При ошибке десериализации возвращается `null` и заполняется сообщение об ошибке.

Фабрика не зависит от контекста выполнения и не должна его учитывать — это задача роутера и хендлера.

Methods

Create(CommandEntry, JsonNode?)

```
public ICommand? Create(CommandEntry entry, JsonNode? argument)
```

Parameters

entry [CommandEntry](#).

argument [JsonNode](#)

Returns

[ICommand](#)

Interface ICommandFactory

Namespace: [Kouki.Mugura.Factories](#)

Assembly: Kouki.Mugura.dll

```
public interface ICommandFactory
```

Methods

Create(CommandEntry, JsonNode?)

```
ICommand? Create(CommandEntry entry, JsonNode? argument)
```

Parameters

entry [CommandEntry](#).

argument [JsonNode](#)

Returns

[ICommand](#)

Namespace Kouki.Mugura.Handlers

Interfaces

[ICommandHandler<TCommand>](#)

Обработчик команды. Возвращает результат в виде JsonNode (универсальный формат).

Interface ICommandHandler<TCommand>

Namespace: [Kouki.Mugura.Handlers](#)

Assembly: Kouki.Mugura.dll

Обработчик команды. Возвращает результат в виде JsonNode (универсальный формат).

```
public interface ICommandHandler<in TCommand> where TCommand : ICommand
```

Type Parameters

TCommand

Тип команды (DTO).

Methods

HandleAsync(TCommand, IExecutionContext?, Cancellation Token)

```
Task<JsonNode> HandleAsync(TCommand command, IExecutionContext? context = null,  
CancellationToken cancellationToken = default)
```

Parameters

command TCommand

context [IExecutionContext](#)

cancellationToken [CancellationToken](#)↗

Returns

[Task](#)↗ <[JsonNode](#)↗ >

Namespace Kouki.Mugura.Registry

Classes

[CommandEntry](#)

Запись параметров команд.

[CommandRegistry](#)

Реестр команд, построенный путём сканирования сборок.

[ValidationRule](#)

Правило валидации для одного свойства DTO команды.

Class CommandEntry

Namespace: [Kouki.Mugura.Registry](#)

Assembly: Kouki.Mugura.dll

Запись параметров команд.

```
public sealed record CommandEntry : IEquatable<CommandEntry>
```

Inheritance

[object](#)  ← CommandEntry

Implements

[IEquatable](#)  <[CommandEntry](#)>

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

CommandEntry(Type, Type, string, string?, bool,
CommandCategory, IReadOnlyList<ValidationRule>, string?)

Запись параметров команд.

```
public CommandEntry(Type CommandType, Type HandlerType, string Name, string? Description,  
bool RequiresConfirmation, CommandCategory Category, IReadOnlyList<ValidationRule>  
ValidationRules, string? Example)
```

Parameters

CommandType [Type](#) 

HandlerType [Type](#) 

Name [string](#) 

Description [string](#) 

RequiresConfirmation [bool](#)

Category [CommandCategory](#)

ValidationRules [IReadOnlyList](#) <[ValidationRule](#)>

Example [string](#)

Properties

Category

```
public CommandCategory Category { get; init; }
```

Property Value

[CommandCategory](#)

CommandType

```
public Type CommandType { get; init; }
```

Property Value

[Type](#)

Description

```
public string? Description { get; init; }
```

Property Value

[string](#)

Example

```
public string? Example { get; init; }
```

Property Value

[string](#)

HandlerType

```
public Type HandlerType { get; init; }
```

Property Value

[Type](#)

Name

```
public string Name { get; init; }
```

Property Value

[string](#)

RequiresConfirmation

```
public bool RequiresConfirmation { get; init; }
```

Property Value

[bool](#)

ValidationRules

```
public IReadOnlyList<ValidationRule> ValidationRules { get; init; }
```

Property Value

[IReadOnlyList](#) <[ValidationRule](#)>

Class CommandRegistry

Namespace: [Kouki.Mugura.Registry](#)

Assembly: Kouki.Mugura.dll

Реестр команд, построенный путём сканирования сборок.

```
public sealed class CommandRegistry
```

Inheritance

[object](#)  ← CommandRegistry

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Remarks

Сканирование выполняется в конструкторе один раз. Находит классы с [CommandAttribute](#), проверяет реализацию [ICommandHandler<TCommand>](#), извлекает DTO и правила валидации. Предоставляет доступ к записям по имени команды и по типу DTO.

Constructors

CommandRegistry(IEnumerable<Assembly>)

```
public CommandRegistry(IEnumerable<Assembly> assemblies)
```

Parameters

assemblies [IEnumerable](#)  <[Assembly](#)  >

Properties

Names

```
public IReadOnlyCollection<string> Names { get; }
```

Property Value

[IReadOnlyCollection](#) <[string](#)>

Methods

GetEntry(string)

```
public CommandEntry? GetEntry(string name)
```

Parameters

name [string](#)

Returns

[CommandEntry](#)

GetEntryByCommandType(Type)

```
public CommandEntry? GetEntryByCommandType(Type commandType)
```

Parameters

commandType [Type](#)

Returns

[CommandEntry](#)

Class ValidationRule

Namespace: [Kouki.Mugura.Registry](#)

Assembly: Kouki.Mugura.dll

Правило валидации для одного свойства DTO команды.

```
public sealed record ValidationRule : IEquatable<ValidationRule>
```

Inheritance

[object](#)  ← ValidationRule

Implements

[IEquatable](#)  <[ValidationRule](#)>

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Remarks

Содержит имя свойства и экземпляр [ValidationAttribute](#) . Используется [DataAnnotationsValidator<TCommand>](#) для проверки.

Constructors

ValidationRule(string, ValidationAttribute)

Правило валидации для одного свойства DTO команды.

```
public ValidationRule(string PropertyName, ValidationAttribute Attribute)
```

Parameters

PropertyName [string](#) 

Attribute [ValidationAttribute](#) 

Remarks

Содержит имя свойства и экземпляр [ValidationAttribute](#)[↗]. Используется [DataAnnotationsValidator<TCommand>](#) для проверки.

Properties

Attribute

```
public ValidationAttribute Attribute { get; init; }
```

Property Value

[ValidationAttribute](#)[↗]

PropertyName

```
public string PropertyName { get; init; }
```

Property Value

[string](#)[↗]

Namespace Kouki.Mugura.Scope

Classes

[ScopeManager](#)

Реализация [IScopeManager](#), создающая DI-скоп для каждого вызова.

Interfaces

[IScopeManager](#)

Interface IScopeManager

Namespace: [Kouki.Mugura.Scope](#)

Assembly: Kouki.Mugura.dll

```
public interface IScopeManager
```

Remarks

TODO: Добавить параметр `configureScope` для передачи контекста в скоуп. Это позволит роутерам настраивать Scoped-сервисы (например, `IContext`) до резолва хендлера.

Methods

`RunInScopeAsync<T>(Func<IServiceProvider, Task<T>>)`

```
Task<T> RunInScopeAsync<T>(Func<IServiceProvider, Task<T>> action)
```

Parameters

action [Func](#) [<IServiceProvider, Task<T>>](#)

Returns

[Task](#) [<T>](#)

Type Parameters

T

Class ScopeManager

Namespace: [Kouki.Mugura.Scope](#)

Assembly: Kouki.Mugura.dll

Реализация [IScopeManager](#), создающая DI-скоп для каждого вызова.

```
public class ScopeManager : IScopeManager
```

Inheritance

[object](#)  ← ScopeManager

Implements

[IScopeManager](#)

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Remarks

Использует [IServiceScopeFactory](#)  для создания скоупа.

Constructors

ScopeManager(IServiceScopeFactory)

```
public ScopeManager(IServiceScopeFactory scopeFactory)
```

Parameters

scopeFactory [IServiceScopeFactory](#) 

Methods

RunInScopeAsync<T>(Func<IServiceProvider, Task<T>>)

```
public Task<T> RunInScopeAsync<T>(Func<IServiceProvider, Task<T>> action)
```

Parameters

action [Func](#)[↗]<[IServiceProvider](#)[↗], [Task](#)[↗]<T>>

Returns

[Task](#)[↗]<T>

Type Parameters

T

Namespace Kouki.Mugura.Validators

Classes

[DataAnnotationsValidator<TCommand>](#)

Валидатор на основе DataAnnotations, использующий правила из [CommandRegistry](#).

Interfaces

[ICommandValidator<TCommand>](#)

Class DataAnnotationsValidator<TCommand>

Namespace: [Kouki.Mugura.Validators](#)

Assembly: Kouki.Mugura.dll

Валидатор на основе DataAnnotations, использующий правила из [CommandRegistry](#).

```
public class DataAnnotationsValidator<TCommand> : ICommandValidator<TCommand> where TCommand : class, ICommand
```

Type Parameters

TCommand

Тип DTO команды (должен быть классом, реализующим [ICommand](#)).

Inheritance

[object](#)  ← DataAnnotationsValidator<TCommand>

Implements

[ICommandValidator](#)<TCommand>

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Remarks

В конструкторе получает [CommandRegistry](#), извлекает для данного типа команды список [Validation Rule](#) и сохраняет их для последующей проверки.

При вызове [TryValidate\(TCommand, out string?\)](#) перебирает все правила и для каждого свойства вызывает метод [IsValid\(object, ValidationContext\)](#) . Если хотя бы одно правило нарушено, возвращает false и сообщение об ошибке.

Кастомные атрибуты валидации (наследники [ValidationAttribute](#) ) работают автоматически — достаточно применить их к свойствам DTO.

Чтобы переопределить валидацию для конкретной команды, зарегистрируйте свою реализацию [ICommandValidator<TCommand>](#) в DI-контейнере после вызова [AddMugura\(IServiceCollection, params Assembly\[\]\)](#).

Constructors

DataAnnotationsValidator(CommandRegistry)

```
public DataAnnotationsValidator(CommandRegistry registry)
```

Parameters

registry [CommandRegistry](#)

Methods

TryValidate(TCommand, out string?)

```
public bool TryValidate(TCommand command, out string? error)
```

Parameters

command TCommand

error [string](#)

Returns

[bool](#)

Interface ICommandValidator<TCommand>

Namespace: [Kouki.Mugura.Validators](#)

Assembly: Kouki.Mugura.dll

```
public interface ICommandValidator<TCommand> where TCommand : ICommand
```

Type Parameters

TCommand

Methods

TryValidate(TCommand, out string?)

```
bool TryValidate(TCommand command, out string? error)
```

Parameters

command TCommand

error [string](#) 

Returns

[bool](#) 

Namespace Shimi.Context

Classes

[BaseExecutionContext](#)

[ContextRegistry](#)

[InMemoryStateStore](#)

Interfaces

[IExecutionContext](#)

Контекст выполнения, содержащий специфичные для окружения реализации сервисов.

Class BaseExecutionContext

Namespace: [Shimi.Context](#)

Assembly: Shimi.dll

```
public class BaseExecutionContext : IExecutionContext
```

Inheritance

[object](#) ← BaseExecutionContext

Implements

[IExecutionContext](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Constructors

BaseExecutionContext(string, IFileSystem, ITerminal, IStateStore)

```
public BaseExecutionContext(string name, IFileSystem fileSystem, ITerminal terminal, IStateStore stateStore)
```

Parameters

name [string](#)

fileSystem [IFileSystem](#)

terminal [ITerminal](#)

stateStore [IStateStore](#)

Properties

FileSystem

Файловая система, специфичная для этого контекста.

```
public IFileSystem FileSystem { get; }
```

Property Value

[IFileSystem](#)

Name

Имя контекста (для идентификации).

```
public string Name { get; }
```

Property Value

[string](#)

StateStore

Хранилище состояния, специфичное для этого контекста.

```
public IStateStore StateStore { get; }
```

Property Value

[IStateStore](#)

Terminal

Терминал, специфичный для этого контекста.

```
public ITerminal Terminal { get; }
```

Property Value

Methods

GetService<T>()

Дополнительные сервисы можно получать по типу (опционально).

```
public T? GetService<T>() where T : class
```

Returns

T

Type Parameters

T

Class ContextRegistry

Namespace: [Shimi.Context](#)

Assembly: Shimi.dll

```
public class ContextRegistry : IContextRegistry
```

Inheritance

[object](#) ← ContextRegistry

Implements

[IContextRegistry](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Constructors

ContextRegistry()

```
public ContextRegistry()
```

Methods

GetContext(string)

Получить контекст по имени (если не найден — возвращает default).

```
public IExecutionContext GetContext(string name)
```

Parameters

name [string](#)

Returns

GetDefaultContext()

Получить контекст по умолчанию.

```
public IExecutionContext GetDefaultContext()
```

Returns

[IExecutionContext](#)

RegisterContext(string, IExecutionContext)

Зарегистрировать контекст.

```
public void RegisterContext(string name, IExecutionContext context)
```

Parameters

name [string](#) 

context [IExecutionContext](#)

SetDefaultContext(IExecutionContext)

Установить контекст по умолчанию.

```
public void SetDefaultContext(IExecutionContext context)
```

Parameters

context [IExecutionContext](#)

SetDefaultContext(string)

```
public void SetDefaultContext(string context)
```

Parameters

context [string](#) 

Interface IExecutionContext

Namespace: [Shimi.Context](#)

Assembly: Shimi.dll

Контекст выполнения, содержащий специфичные для окружения реализации сервисов.

```
public interface IExecutionContext
```

Properties

FileSystem

Файловая система, специфичная для этого контекста.

```
IFFileSystem FileSystem { get; }
```

Property Value

[IFFileSystem](#)

Name

Имя контекста (для идентификации).

```
string Name { get; }
```

Property Value

[string](#)

StateStore

Хранилище состояния, специфичное для этого контекста.

```
IStateStore StateStore { get; }
```

Property Value

[IStateStore](#)

Terminal

Терминал, специфичный для этого контекста.

```
ITerminal Terminal { get; }
```

Property Value

[ITerminal](#)

Methods

GetService<T>()

Дополнительные сервисы можно получать по типу (опционально).

```
T? GetService<T>() where T : class
```

Returns

T

Type Parameters

T

Class InMemoryStateStore

Namespace: [Shimi.Context](#)

Assembly: Shimi.dll

```
public class InMemoryStateStore : IStateStore
```

Inheritance

[object](#) ← InMemoryStateStore

Implements

[IStateStore](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

GetAsync<T>(string, CancellationToken)

```
public Task<T?> GetAsync<T>(string key, CancellationToken ct = default)
```

Parameters

key [string](#)

ct [CancellationToken](#)

Returns

[Task](#) <T>

Type Parameters

T

RemoveAsync(string, CancellationToken)

```
public Task RemoveAsync(string key, CancellationToken ct = default)
```

Parameters

key [string](#)

ct [CancellationToken](#)

Returns

[Task](#)

SetAsync<T>(string, T, CancellationToken)

```
public Task SetAsync<T>(string key, T value, CancellationToken ct = default)
```

Parameters

key [string](#)

value T

ct [CancellationToken](#)

Returns

[Task](#)

Type Parameters

T

Namespace Shimi.Context.Dependency Injection

Classes

[ServiceCollectionExtensions](#)

Class ServiceCollectionExtensions

Namespace: [Shimi.Context.DependencyInjection](#)

Assembly: Shimi.dll

```
public static class ServiceCollectionExtensions
```

Inheritance

[object](#)  ← ServiceCollectionExtensions

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

AddShimiContext(IServiceCollection)

Регистрирует базовые сервисы Shimi: контекстный реестр и хранилище в памяти.

```
public static IServiceCollection AddShimiContext(this IServiceCollection services)
```

Parameters

services [IServiceCollection](#) 

Коллекция сервисов.

Returns

[IServiceCollection](#) 

IServiceCollection для цепочки вызовов.

Namespace Shimi.ServiceAbstractions

Interfaces

[IFileSystem](#)

TODO

[IJsonFileSystem](#)

Расширение для работы с JSON-файлами.

[IStateStore](#)

TODO

[ITelemetryLogger](#)

[ITerminal](#)

TODO

[ITextFileSystem](#)

Interface IFileSystem

Namespace: [Shimi.ServiceAbstractions](#)

Assembly: Shimi.dll

TODO

```
public interface IFileSystem
```

Methods

ExistsAsync(string, CancellationToken)

```
Task<bool> ExistsAsync(string path, CancellationToken ct = default)
```

Parameters

path [string](#)

ct [CancellationToken](#)

Returns

[Task](#) <[bool](#)>

GetTreeAsync(string, bool, CancellationToken)

```
Task<string> GetTreeAsync(string path, bool showFiles, CancellationToken ct = default)
```

Parameters

path [string](#)

showFiles [bool](#)

ct [CancellationToken](#)

Returns

[Task](#) <[string](#)>

ReadBytesAsync(string, CancellationToken)

```
Task<byte[]> ReadBytesAsync(string path, CancellationToken ct = default)
```

Parameters

path [string](#)

ct [CancellationToken](#)

Returns

[Task](#) <[byte](#)[]>

WriteBytesAsync(string, byte[], CancellationToken)

```
Task WriteBytesAsync(string path, byte[] data, CancellationToken ct = default)
```

Parameters

path [string](#)

data [byte](#)[]

ct [CancellationToken](#)

Returns

[Task](#)

Interface IJsonFileSystem

Namespace: [Shimi.ServiceAbstractions](#)

Assembly: Shimi.dll

Расширение для работы с JSON-файлами.

```
public interface IJsonFileSystem : IFileSystem
```

Inherited Members

[IFileSystem.ReadBytesAsync\(string, CancellationToken\)](#) ,
[IFileSystem.WriteBytesAsync\(string, byte\[\], CancellationToken\)](#) ,
[IFileSystem.ExistsAsync\(string, CancellationToken\)](#) ,
[IFileSystem.GetTreeAsync\(string, bool, CancellationToken\)](#)

Methods

ReadJsonAsync<T>(string, JsonSerializerOptions?,
CancellationToken)

```
Task<T> ReadJsonAsync<T>(string path, JsonSerializerOptions? options = null,  
CancellationToken ct = default)
```

Parameters

path [string](#)[↗]

options [JsonSerializerOptions](#)[↗]

ct [CancellationToken](#)[↗]

Returns

[Task](#)[↗]<T>

Type Parameters

T

WriteJsonAsync<T>(string, T, JsonSerializerOptions?, CancellationToken)

```
Task WriteJsonAsync<T>(string path, T data, JsonSerializerOptions? options = null,
    CancellationToken ct = default)
```

Parameters

path [string](#)

data T

options [JsonSerializerOptions](#)

ct [CancellationToken](#)

Returns

[Task](#)

Type Parameters

T

Interface IStateStore

Namespace: [Shimi.ServiceAbstractions](#)

Assembly: Shimi.dll

TODO

```
public interface IStateStore
```

Methods

GetAsync<T>(string, CancellationToken)

```
Task<T?> GetAsync<T>(string key, CancellationToken ct = default)
```

Parameters

key [string](#) 

ct [CancellationToken](#) 

Returns

[Task](#)  <T>

Type Parameters

T

RemoveAsync(string, CancellationToken)

```
Task RemoveAsync(string key, CancellationToken ct = default)
```

Parameters

key [string](#) 

ct [CancellationToken](#)

Returns

[Task](#)

SetAsync<T>(string, T, CancellationToken)

```
Task SetAsync<T>(string key, T value, CancellationToken ct = default)
```

Parameters

key [string](#)

value T

ct [CancellationToken](#)

Returns

[Task](#)

Type Parameters

T

Interface ITelemetryLogger

Namespace: [Shimi.ServiceAbstractions](#)

Assembly: Shimi.dll

```
public interface ITelemetryLogger
```

Methods

LogError(string, Exception?)

```
void LogError(string message, Exception? ex = null)
```

Parameters

message [string](#) 

ex [Exception](#) 

LogEvent(string, object?)

```
void LogEvent(string name, object? data = null)
```

Parameters

name [string](#) 

data [object](#) 

Interface ITerminal

Namespace: [Shimi.ServiceAbstractions](#)

Assembly: Shimi.dll

TODO

```
public interface ITerminal
```

Methods

RunCommandAsync(string, string?, CancellationToken)

```
Task<string> RunCommandAsync(string command, string? workingDirectory = null,  
CancellationToken ct = default)
```

Parameters

command [string](#) 

workingDirectory [string](#) 

ct [CancellationToken](#) 

Returns

[Task](#)  <[string](#)  >

Interface ITextFileSystem

Namespace: [Shimi.ServiceAbstractions](#)

Assembly: Shimi.dll

```
public interface ITextFileSystem : IFileSystem
```

Inherited Members

[IFileSystem.ReadBytesAsync\(string, CancellationToken\)](#) ,
[IFileSystem.WriteBytesAsync\(string, byte\[\], CancellationToken\)](#) ,
[IFileSystem.ExistsAsync\(string, CancellationToken\)](#) ,
[IFileSystem.GetTreeAsync\(string, bool, CancellationToken\)](#)

Methods

AppendTextAsync(string, string, Encoding?, CancellationToken)

```
Task AppendTextAsync(string path, string content, Encoding? encoding = null,  
CancellationToken ct = default)
```

Parameters

path [string](#) 

content [string](#) 

encoding [Encoding](#) 

ct [CancellationToken](#) 

Returns

[Task](#) 

ReadTextAsync(string, Encoding?, CancellationToken)

```
Task<string> ReadTextAsync(string path, Encoding? encoding = null, CancellationToken ct = default)
```

Parameters

path [string](#)

encoding [Encoding](#)

ct [CancellationToken](#)

Returns

[Task](#) <[string](#)>

WriteTextAsync(string, string, Encoding?, CancellationToken)

```
Task WriteTextAsync(string path, string content, Encoding? encoding = null, CancellationToken ct = default)
```

Parameters

path [string](#)

content [string](#)

encoding [Encoding](#)

ct [CancellationToken](#)

Returns

[Task](#)